What is Pandas?

Python library used for working with structured data sets (tables). It has functions for analyzing, cleaning, exploring, and manipulating data. Similar to data tables in SQL and Excel.

0 install pandas

```
!pip install pandas
```

```
Requirement already satisfied: pandas in /usr/local/lib/python3.12/d
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/pytho
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/pyth
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12
```

```
# import pandas
import pandas as pd
```

```
pd.__version__ # check installed pandas version
```

```
'2.2.2'
```

What is Dataframe?

It's a structured data constructed with rows and columns, similar to a SQL tables or Excel tables

1 Create Dataframe

```
# using a List
df = pd.DataFrame([11,22,33], columns=['Col_Name'])
print(df)
```

```
   Col_Name
0        11
1        22
2        33
```

```
print(type(df)) # check data type
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
# using Dictionary of Lists
data = {
    'Name': ['Madhav', 'Vishakha', 'Lalita', 'Rishabh'],
    'Age': [16,17,18,19],
    'Salary': [90000, 70000, 50000, 30000]
}
```

```
df = pd.DataFrame(data)
print(df)
```

```
       Name  Age  Salary
0    Madhav   16   90000
1  Vishakha   17   70000
2    Lalita   18   50000
3   Rishabh   19   30000
```

```
print(type(df))
```

```
<class 'pandas.core.frame.DataFrame'>
```

2 Basic Dataframe Understanding

```
df.head(2) # top rows
```

	Name	Age	Salary
0	Madhav	16	90000
1	Vishakha	17	70000

```
df.tail(2) # last rows
```

	Name	Age	Salary
2	Lalita	18	50000
3	Rishabh	19	30000

```
df.shape # returns a tuple containing the shape of the DataFrame –
```

```
(4, 3)
```

```
df.columns # list of column names in a dataframe
```

```
Index(['Name', 'Age', 'Salary'], dtype='object')
```

```
df.rename(columns={'Salary': 'Monthly_Salary'}, inplace=True) # ren
```

```
df
```

	Name	Age	Monthly_Salary
0	Madhav	16	90000
1	Vishakha	17	70000
2	Lalita	18	50000
3	Rishabh	19	30000

```
df.info()  # info method prints information about the DataFrame
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4 entries, 0 to 3
Data columns (total 3 columns):
 #   Column          Non-Null Count  Dtype 
---  ------          --------------  ----- 
 0   Name            4 non-null      object
 1   Age             4 non-null      int64 
 2   Monthly_Salary  4 non-null      int64 
dtypes: int64(2), object(1)
memory usage: 228.0+ bytes
```

```
df.describe() # describe method generates descriptive statistics of
```

	Age	Monthly_Salary
count	4.000000	4.000000
mean	17.500000	60000.000000
std	1.290994	25819.888975
min	16.000000	30000.000000
25%	16.750000	45000.000000
50%	17.500000	60000.000000
75%	18.250000	75000.000000
max	19.000000	90000.000000

3 Save and Load data from csv

```
df.to_csv('Test_data.csv', index=False) # save file - export data f
```

```
load_df= pd.read_csv('Test_data.csv') # load file - import datafram
load_df
```

	Name	Age	Monthly_Salary
0	Madhav	16	90000
1	Vishakha	17	70000
2	Lalita	18	50000
3	Rishabh	19	30000

4 Rows & Columns - Selection

```
# select single column
df[['Name']]
```

	Name
0	Madhav
1	Vishakha
2	Lalita
3	Rishabh

```
# select multiple columns
df[['Name', 'Monthly_Salary']]
```

	Name	Monthly_Salary
0	Madhav	90000
1	Vishakha	70000
2	Lalita	50000
3	Rishabh	30000

```
# select single row || loc - label based index
df.loc[df.Name=='Madhav']
```

	Name	Age	Monthly_Salary
0	Madhav	16	90000

```
# select multiple rows || loc - label based index
df.loc[(df.Name=='Madhav') & (df.Monthly_Salary>=50000)]
```

	Name	Age	Monthly_Salary
0	Madhav	16	90000

```
df.loc[0:2]
```

	Name	Age	Monthly_Salary
0	Madhav	16	90000
1	Vishakha	17	70000
2	Lalita	18	50000

```
# Select Rows || iloc - index-value based
# df.iloc[0]
df.iloc[0:2] # [start:stop:step]
```

	Name	Age	Monthly_Salary
0	Madhav	16	90000
1	Vishakha	17	70000

5 Filter Dataframe - Filtering by column values

```
df
```

	Name	Age	Monthly_Salary
0	Madhav	16	90000
1	Vishakha	17	70000
2	Lalita	18	50000
3	Rishabh	19	30000

```
df_age_filter = df[df['Age'] >= 18] # filter and store dataframe in
```

```
df_age_filter
```

	Name	Age	Monthly_Salary
2	Lalita	18	50000
3	Rishabh	19	30000

```
df[(df['Age'] >= 18) & (df['Monthly_Salary'] >= 50000)] # multiple
```

	Name	Age	Monthly_Salary
2	Lalita	18	50000

```
df.where(df['Age'] >= 18) # where function replace values in a Data
```

	Name	Age	Monthly_Salary
0	NaN	NaN	NaN
1	NaN	NaN	NaN
2	Lalita	18.0	50000.0
3	Rishabh	19.0	30000.0

```
df.where(df['Age'] >= 18, other = 'Not Eligible')
```

	Name	Age	Monthly_Salary
0	Not Eligible	Not Eligible	Not Eligible
1	Not Eligible	Not Eligible	Not Eligible
2	Lalita	18	50000
3	Rishabh	19	30000

6 Rows and Columns - Operation (Add, update, delete)

```
df
```

	Name	Age	Monthly_Salary
0	Madhav	16	90000
1	Vishakha	17	70000
2	Lalita	18	50000
3	Rishabh	19	30000

```
# Create a new column
df['Team'] = ['CEO', 'HR', 'CTO', 'DA']
df
```

	Name	Age	Monthly_Salary	Team
0	Madhav	16	90000	CEO
1	Vishakha	17	70000	HR
2	Lalita	18	50000	CTO
3	Rishabh	19	30000	DA

```
# Add new columns using existing column/s
df['Bonus'] = df['Monthly_Salary'] * 0.20
df
```

	Name	Age	Monthly_Salary	Team	Bonus
0	Madhav	16	90000	CEO	18000.0
1	Vishakha	17	70000	HR	14000.0
2	Lalita	18	50000	CTO	10000.0
3	Rishabh	19	30000	DA	6000.0

```
# Add new row - at the end of dataframe
df.loc[len(df)] = ['ABC', 21, 21000, 'IT', 2000]
df
```

	Name	Age	Monthly_Salary	Team	Bonus
0	Madhav	16	90000	CEO	18000.0
1	Vishakha	17	70000	HR	14000.0
2	Lalita	18	50000	CTO	10000.0
3	Rishabh	19	30000	DA	6000.0
4	ABC	21	21000	IT	2000.0

```
# update value in dataframe using index-name
df.loc[0, 'Monthly_Salary'] = 95000
df
```

	Name	Age	Monthly_Salary	Team	Bonus
0	Madhav	16	95000	CEO	18000.0
1	Vishakha	17	70000	HR	14000.0
2	Lalita	18	50000	CTO	10000.0
3	Rishabh	19	30000	DA	6000.0
4	ABC	21	21000	IT	2000.0

```
# update value in dataframe using column-value
df.loc[df.Name=='Madhav','Monthly_Salary'] = 90000
df
```

	Name	Age	Monthly_Salary	Team	Bonus
0	Madhav	16	90000	CEO	18000.0
1	Vishakha	17	70000	HR	14000.0
2	Lalita	18	50000	CTO	10000.0
3	Rishabh	19	30000	DA	6000.0
4	ABC	21	21000	IT	2000.0

```
# delete value - rows and columns
```

```
# delete row using column-value filter
df = df.drop(df[df.Name == 'ABC'].index) # delete row, axis= 0
```

```
# delete row using index-name filter
df.drop(1, axis=0)
```

	Name	Age	Monthly_Salary	Team	Bonus
0	Madhav	16	90000	CEO	18000.0
2	Lalita	18	50000	CTO	10000.0
3	Rishabh	19	30000	DA	6000.0

```
df.drop('Bonus', axis=1, inplace=True) # delete one column
# df.drop(['Bonus', 'Team'], axis=1, inplace=True) # delete multipl
```

```
df
```

	Name	Age	Monthly_Salary	Team
0	Madhav	16	90000	CEO
1	Vishakha	17	70000	HR
2	Lalita	18	50000	CTO
3	Rishabh	19	30000	DA

```
df.rename(columns={'Monthly_Salary': 'Salary'}, inplace=True) # ren
```

```
# sort values - order values in dataframe by asc or desc order
df.sort_values('Salary') #ascending order, by default
```

	Name	Age	Salary	Team
3	Rishabh	19	30000	DA
2	Lalita	18	50000	CTO
1	Vishakha	17	70000	HR
0	Madhav	16	90000	CEO

```
# sort values in descending order
df.sort_values('Salary', ascending=False)
```

	Name	Age	Salary	Team
0	Madhav	16	90000	CEO
1	Vishakha	17	70000	HR
2	Lalita	18	50000	CTO
3	Rishabh	19	30000	DA

7 working with date value

```
# create a date column - date of joining
df['DOJ'] = ['2024-01-01', '2024-01-15', '2024-03-28', '2024-03-03'
df
```

	Name	Age	Salary	Team	DOJ
0	Madhav	16	90000	CEO	2024-01-01
1	Vishakha	17	70000	HR	2024-01-15
2	Lalita	18	50000	CTO	2024-03-28
3	Rishabh	19	30000	DA	2024-03-03

```
df['DOJ'].dtype # object-type value
```

```
dtype('O')
```

```
df['DOJ'] = pd.to_datetime(df['DOJ']) # change to date-time type
```

```
df['DOJ'].dtype # date-time type
```

```
dtype('<M8[ns]')
```

```
df1 = df
```

```
# creating a new column with incorrect date format
df1['DOJ2'] = ['01-01-2025', '15-01-2025', '28-03-2025', '03-03-202
df1
```

	Name	Age	Salary	Team	DOJ	DOJ2
0	Madhav	16	90000	CEO	2024-01-01	01-01-2025
1	Vishakha	17	70000	HR	2024-01-15	15-01-2025
2	Lalita	18	50000	CTO	2024-03-28	28-03-2025
3	Rishabh	19	30000	DA	2024-03-03	03-03-2025

```
df1['DOJ2'].dtype  # object-type value
```

```
dtype('O')
```

```
df1['DOJ2'] = pd.to_datetime(df1['DOJ2'], format = '%d-%m-%Y') # ch
```

```
df1['DOJ2'].dtype
```

```
dtype('<M8[ns]')
```

```
df = df.drop('DOJ2', axis=1)
df
```

	Name	Age	Salary	Team	DOJ
0	Madhav	16	90000	CEO	2024-01-01
1	Vishakha	17	70000	HR	2024-01-15
2	Lalita	18	50000	CTO	2024-03-28
3	Rishabh	19	30000	DA	2024-03-03

```
# extract year, month, week, day
df['DOJ'].dt.year
df['DOJ'].dt.month
df['DOJ'].dt.day
df['DOJ'].dt.day_name()
```

```
0      Monday
1      Monday
2    Thursday
3      Sunday
Name: DOJ, dtype: object
```

```
# create new column using month extract function from DOJ column
df['Month'] = df['DOJ'].dt.month
```

```
df
```

	Name	Age	Salary	Team	DOJ	Month
0	Madhav	16	90000	CEO	2024-01-01	1
1	Vishakha	17	70000	HR	2024-01-15	1
2	Lalita	18	50000	CTO	2024-03-28	3
3	Rishabh	19	30000	DA	2024-03-03	3

```
df['DOJ'] + pd.Timedelta(days=90) # add 90 days to DOJ column
```

```
0   2024-03-31
1   2024-04-14
2   2024-06-26
3   2024-06-01
Name: DOJ, dtype: datetime64[ns]
```

8 Handling Missing Values

```
df
```

	Name	Age	Salary	Team	DOJ	Month
0	Madhav	16	90000	CEO	2024-01-01	1
1	Vishakha	17	70000	HR	2024-01-15	1
2	Lalita	18	50000	CTO	2024-03-28	3
3	Rishabh	19	30000	DA	2024-03-03	3

```
df.isnull() #  Detect missing values
```

	Name	Age	Salary	Team	DOJ	Month
0	False	False	False	False	False	False
1	False	False	False	False	False	False
2	False	False	False	False	False	False
3	False	False	False	False	False	False

```
import numpy as np  # to create null values below
```

```
df.loc[df.Name=='Rishabh', 'Salary'] = np.nan # adding a null value
```

```
df
```

	Name	Age	Salary	Team	DOJ	Month
0	Madhav	16	90000.0	CEO	2024-01-01	1
1	Vishakha	17	70000.0	HR	2024-01-15	1
2	Lalita	18	50000.0	CTO	2024-03-28	3
3	Rishabh	19	NaN	DA	2024-03-03	3

```
df.isnull()
```

	Name	Age	Salary	Team	DOJ	Month
0	False	False	False	False	False	False
1	False	False	False	False	False	False
2	False	False	False	False	False	False
3	False	False	True	False	False	False

```
df.isnull().sum() # count of null values by columns
```

```
Name      0
Age       0
Salary    1
Team      0
DOJ       0
Month     0
dtype: int64
```

```
df.fillna(0) # fill null values with 0
```

	Name	Age	Salary	Team	DOJ	Month
0	Madhav	16	90000.0	CEO	2024-01-01	1
1	Vishakha	17	70000.0	HR	2024-01-15	1
2	Lalita	18	50000.0	CTO	2024-03-28	3
3	Rishabh	19	0.0	DA	2024-03-03	3

```
df.loc[df.Name=='Rishabh', 'Salary'] = 30000
```

```
df
```

	Name	Age	Salary	Team	DOJ	Month
0	Madhav	16	90000.0	CEO	2024-01-01	1
1	Vishakha	17	70000.0	HR	2024-01-15	1
2	Lalita	18	50000.0	CTO	2024-03-28	3
3	Rishabh	19	30000.0	DA	2024-03-03	3

9 Aggregation and Group By

```
df['Month'].value_counts() # frequency of values in month column
```

```
Month
1    2
3    2
Name: count, dtype: int64
```

```
df[df['Month']==1].value_counts() # frequency of values in month cc
```

```
Name      Age  Salary   Team  DOJ         Month
Madhav    16   90000.0  CEO   2024-01-01  1        1
Vishakha  17   70000.0  HR    2024-01-15  1        1
Name: count, dtype: int64
```

```
# aggregation based on group by
df.groupby('Month')['Salary'].sum() # sum of salary by month
```

```
Month
1    160000.0
3     80000.0
Name: Salary, dtype: float64
```

```
# different aggregation on different columns
df.groupby('Month').agg({'Salary': 'mean', 'Name': 'count'})
```

Month	Salary	Name
1	80000.0	2
3	40000.0	2

10 Concatenate and Merge Dataframe (JOINS)

```
df1 = pd.DataFrame({'ID':[1,2,3],'Name':['A','B','C']})
df1
```

	ID	Name
0	1	A
1	2	B
2	3	C

```
df2 = pd.DataFrame({'ID':[1,2,2,4],'Score':[88,96,77,79]})
df2
```

	ID	Score
0	1	88
1	2	96
2	2	77
3	4	79

What is Concatenate:

Combine objects along a particular axis.

```
# Concatenate: vertical / row level / top on top
pd.concat([df1, df2], axis=0)
```

	ID	Name	Score
0	1	A	NaN
1	2	B	NaN
2	3	C	NaN
0	1	NaN	88.0
1	2	NaN	96.0
2	2	NaN	77.0
3	4	NaN	79.0

```
# Concatenate: horizontal / column level / side on side
pd.concat([df1, df2], axis=1)
```

	ID	Name	ID	Score
0	1.0	A	1	88
1	2.0	B	2	96
2	3.0	C	2	77
3	NaN	NaN	4	79

Merge in Pandas

Performs join operations similar to relational databases like SQL JOINS

```
# merge = join
pd.merge(df1, df2, how='inner', on='ID')
```

	ID	Name	Score
0	1	A	88
1	2	B	96
2	2	B	77

```
pd.merge(df1, df2, how='inner', left_on='ID', right_on='ID')
```

	ID	Name	Score
0	1	A	88
1	2	B	96
2	2	B	77

11 AI and ChatGPT

```
df
```

	Name	Age	Salary	Team	DOJ	Month
0	Madhav	16	90000.0	CEO	2024-01-01	1
1	Vishakha	17	70000.0	HR	2024-01-15	1
2	Lalita	18	50000.0	CTO	2024-03-28	3
3	Rishabh	19	30000.0	DA	2024-03-03	3

```
# Prompt to filter salary > 70000 and January employees
```

```
# Method-1: basic filter
df[(df['Month'] == 1) & (df['Salary'] >= 70000)]
```

	Name	Age	Salary	Team	DOJ	Month
0	Madhav	16	90000.0	CEO	2024-01-01	1
1	Vishakha	17	70000.0	HR	2024-01-15	1

```
# Method-2: query method
df.query("Month == 1 and Salary >= 70000")
```

	Name	Age	Salary	Team	DOJ	Month
0	Madhav	16	90000.0	CEO	2024-01-01	1
1	Vishakha	17	70000.0	HR	2024-01-15	1

Home work: Create a dataframe to practice pivot table in pandas (using AI prompt)

Download Pandas Code Notes:

https://www.rishabhnmishra.com/notes

```
Start coding or generate with AI.
```